Stack Overflows in RTOS-Based Designs – Part 1

This article is written by Jean J. Labrosse, RTOS Expert.

Each task in an RTOS-based application requires its own stack, the size of which depends on the task's requirements (e.g., function call nesting, arguments passed to functions, local variables, etc.).

To avoid stack overflows, a developer needs to over-allocate stack space, yet not too much, to avoid wasting RAM.

What are Stack Overflows?

Just so that we are on the same page, below is a description of what a stack overflow is. For the sake of discussion, it's assumed here that stacks grow from high-memory to low-memory. Of course, the same issue occurs when the stack grows in the other direction. Refer to Figure 1.

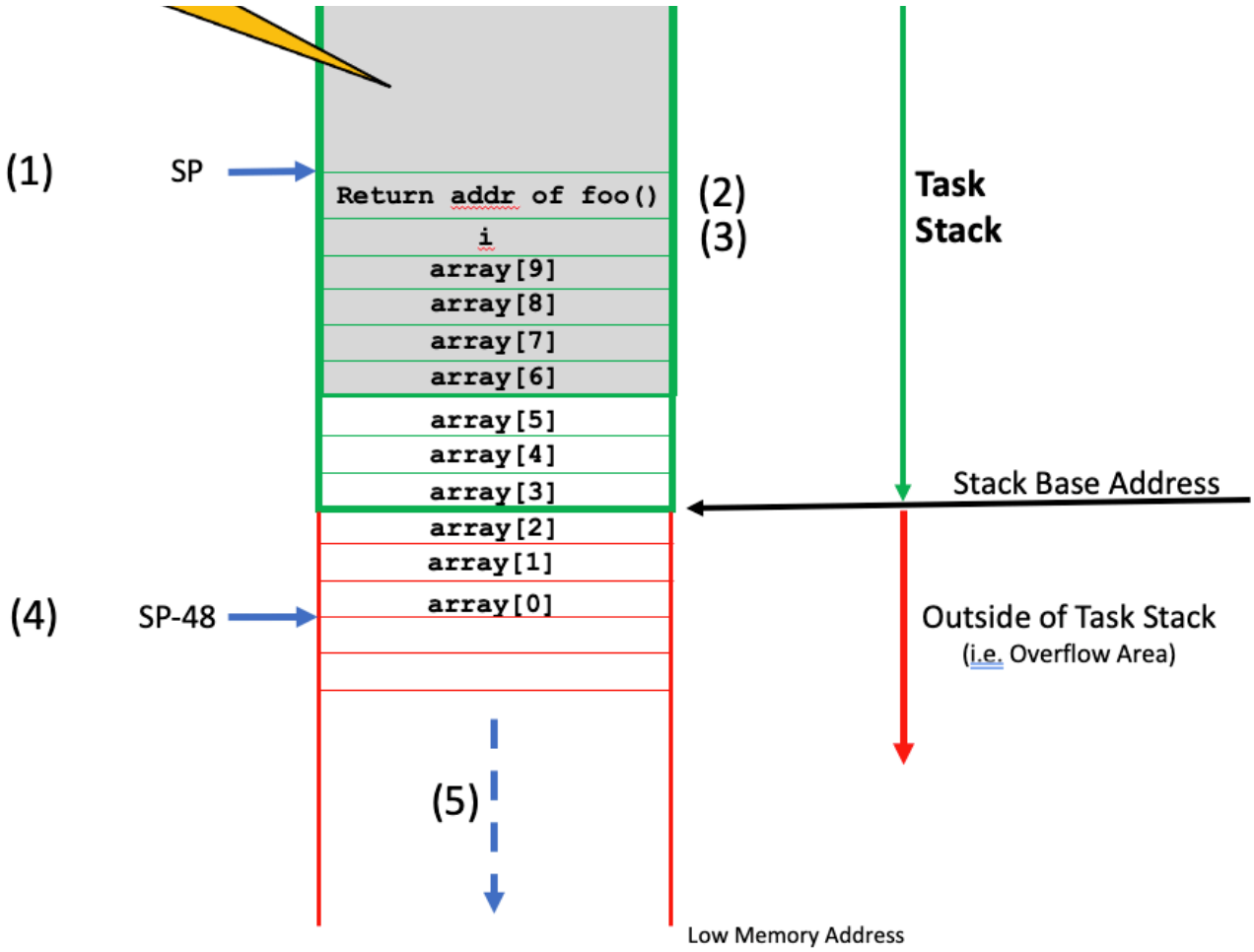

Figure 1 – Stack Overflow

(1) The CPU's Stack Pointer (SP) register points somewhere inside the stack space allocated for a task. The task is about to call the function foo()as shown below.

```
void  foo (void)
{
    int  i;
    int  array[10];

    :
    :
    // Code
}
```

(3) The compiler then adjusts the SP to accommodate for local variables. Unfortunately, at this point, we overflowed the stack (the SP points outside the storage area assigned for the stack) and just about anything foo() does will corrupt whatever data is beyond the stack base. In fact, depending on the code flow, the array might never be used, in which case the problem would not be immediately apparent. However, if foo() calls another function, there is a high likelihood that will cause something outside the stack to be touched.

(4) So, when foo() starts to execute code, the stack pointer has an offset of 48 bytes from where it was prior to calling foo() (assuming a stack entry is 4 bytes wide).

(5) We typically don't know what resides here. It could be the stack of another task, it could be variables, data structures or an array used by the application. Overwriting whatever resides here can cause strange behaviors: values computed by another task may not be what you expected and could cause decisions in your code to take the wrong path, or your system may work fine under normal conditions but then fail. We just don't know and it's actually quite difficult to predict. In fact, the behavior can change each time you make changes to your code.

Whenever someone mentions that his or her application behaves "strangely," insufficient stack size is the first thing that comes to mind.

How do you determine the size of a task stack?

The size of the stack required by a task is application specific but, it's possible to manually figure out the stack space needed by adding up:

1) The memory required by all function call nesting. For each function call hierarchy level:

- Depending on the CPU architecture, one pointer for the return address of a function call. Some CPUs actually save the return address in a special register reserved for that purpose (often called the Link Register or, LR). However, if the function calls another function, the LR must be saved by the caller so, it might be wise to assume that the LR is pushed onto the stack anyway.
- The memory required by the arguments passed in those function calls. Arguments are often passed in CPU registers but again, if a function calls other functions the register contents will be saved onto the stack anyway. I would thus

- Storage of local variables for those functions
- Additional stack space for state saving operations inside the functions

The IAR Linker has a neat feature which helps determine how much stack space each task will require.

As shown in Figure 2, you simply check boxes in the Linker's configuration, build your code and examine the link map (.MAP file) to reveal the call stack depth (in bytes) for each function in your application.

You then note the task's call stack size for each of your tasks because, we still need to add a couple of numbers to determine maximum stack size.

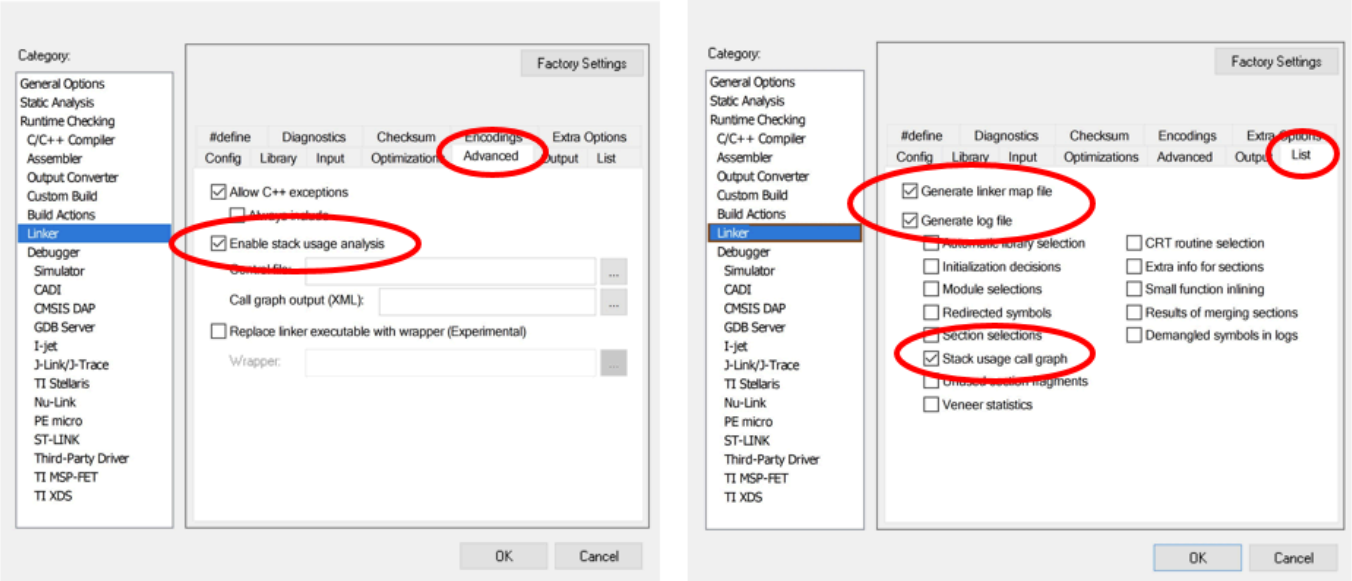

Figure 2 – IAR Linker Configuration Options

2) The storage for a full CPU context (depends on the CPU) plus FPU registers as needed

3) The storage of another full CPU context for each nested ISR (if the CPU doesn't have a separate stack to handle ISRs)

4) The stack space needed for local variables used by those ISRs.

In fact, in most embedded applications, it's desirable to keep run-time stack usage below the 70% mark. You can certainly be more conservative as needed by your requirements.

$$Task_{TotalStackSize} = 1.33\ x\ (Task_{CallStackSize} + CPU_{Context} + FPU_{Context})$$

The stack usage calculation assumes that the exact path of the code is known at all time, which is not always possible.

Specifically, when calling a function such as **printf()**, it might be difficult or nearly impossible to even guess just how much stack space **printf()** will require. Also indirect function calls through tables of function pointers could be problematic.

Finally, you should avoid writing recursive code because stack usage is typically non-deterministic (or difficult to determine) with this type of code.

Generally speaking, you'd start with a fairly large stack space, run the application under worse case conditions and monitor the stack usage at run-time.

Some RTOSs (specifically uC/OS-III and Cesium/OS3) allow you to monitor stack usage at run-time which you can then display using the RTOS awareness capability built into IAR's C-SPY.

Want to learn more?

Check out **Part 2, Detecting Stack Overflows in RTOS-Based Designs**, or access the on-demand webinar, **Tips and hints for better debugging your RTOS-based application**.

About the author

This article is part of a series on the topic of developing applications with RTOS.

You might also be interested in:

World leader of software and services for embedded systems development.

Products

Overview

Architectures

Industries

Requirements

Investors

About IAR Systems Group

About

News